

DVA494

Lab 4: Sequential Binary Multiplier

Spring 2026

1 Introduction and Objectives

In modern digital design, complex systems are rarely designed as a single, massive Finite State Machine (FSM). As the number of flip-flops (n) increases, the state space grows exponentially (2^n), making conventional design techniques impractical.

To manage this complexity, systems are partitioned into two distinct units:

1. **Data Path (Execution Unit):** Contains the components that store and process data, such as registers, adders, and multiplexers.
2. **Control Path (Controller):** A dedicated FSM that generates command signals to orchestrate the Data Path operations in the correct sequence.

In this lab, you will implement a 4-bit Sequential Multiplier using this modular approach. While we are focusing on a 4-bit implementation, the architecture is scalable; an n -bit multiplier can be created by simply increasing register and adder widths without altering the fundamental controller logic.

Learning Objectives

- Master the "Divide and Conquer" design methodology using Data Paths and Control Paths.
- Implement a sequential multiplication algorithm using shifted additions.
- Integrate hardware peripherals (Switches, Buttons, and 7-Segment Displays) on the Basys 3 board to create a complete user application.

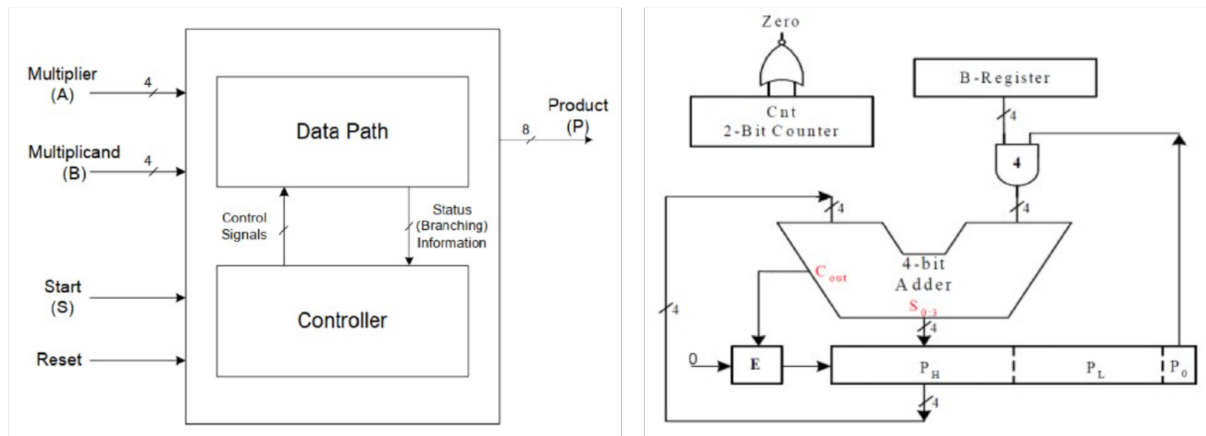
2 Design Specification

Your design will consist of a hierarchical VHDL structure. You must implement the sequential multiplication algorithm (Add-and-Shift) as detailed in Lecture 8.

1. The Multiplier Core (Sequential_Multiplier)

This module encapsulates the FSM and the arithmetic logic. It must interact with the environment using the following signals:

```
1 entity Sequential_Multiplier is
2   Generic ( N : integer := 4 );
3   Port (
4       clk       : in  STD_LOGIC;
5       resetn    : in  STD_LOGIC;
6       start     : in  STD_LOGIC;
7       A         : in  STD_LOGIC_VECTOR(N-1 downto 0); -- Multiplier
8       B         : in  STD_LOGIC_VECTOR(N-1 downto 0); -- Multiplicand
9       P         : out STD_LOGIC_VECTOR(2*N-1 downto 0)
10  );
11 end Sequential_Multiplier;
```



Data Path Architecture

The data path must implement the following key registers and components to handle the multiplication logic:

- **B-register (Multiplicand):** A 4-bit register that stores the value of B for the duration of the operation.
- **P-register (Partial Product):** An 8-bit register split into two 4-bit segments:
 - P_H (**High Byte**): Initially set to 0; it stores the accumulating partial product.
 - P_L (**Low Byte**): Initially loaded with operand A (the multiplier).
- **E-register:** A 1-bit register used to store the adder carry-out; it behaves as the MSB of the product during shifts.
- **Cnt (Iteration Counter):** A down-counter initialized to $n = 4$ to track when all multiplier bits have been processed.

Operational/Control Logic

The final product is represented by the concatenated bits (E, P_H, P_L) . The multiplication follows these steps:

1. **Inspect LSB:** Check the least significant bit of the P-register ($P_L(0)$).
2. **Add:** If $P_L(0) = 1$, add the B-register to P_H . Store the result in P_H and the carry-out in E .
3. **Shift:** Shift the combined $\{E, P_H, P_L\}$ registers one bit to the right.
4. **Loop:** Decrement Cnt and repeat until the counter reaches zero.

Refer to the FSM/ASM discussed in Lecture 8 for the implementation of the controller.

2. The Top-Level Application (Top_Multiplier)

To test your design on the Basys 3 board, you will wrap the multiplier core in a top-level module that maps internal signals to physical hardware:

- **Data Entry:** Use the 8 on-board switches ($sw[7:0]$) to provide operands A and B (4 bits each).
- **Control:** Use the center push-button (btnC) as the start signal.
- **Visualization:** The 8-bit result must be displayed in decimal format on the **7-Segment Display**.

```

1  entity Top_Multiplier is
2      Port (
3          clk      : in  STD_LOGIC;  -- System clock (100 MHz)
4          btnC     : in  STD_LOGIC;  -- Start button
5          sw       : in  STD_LOGIC_VECTOR(7 downto 0); -- Input switches
6          led      : out STD_LOGIC_VECTOR(7 downto 0); -- LEDs to show switch state
7          an       : out STD_LOGIC_VECTOR(3 downto 0); -- 7-segment anodes
8          seg      : out STD_LOGIC_VECTOR(6 downto 0) -- 7-segment segments
9      );
10 end Top_Multiplier;

```